

is employed to construct the following practical loss function to train the neural network solution $U(\mathbf{x}; \theta)$,

$$J_{R,M}(\theta) := \frac{w_I}{|X(\Omega)|} \sum_{\mathbf{x} \in X(\Omega)} \left(\frac{1}{2} |\nabla U(\mathbf{x}; \theta)|^2 - f(\mathbf{x})U(\mathbf{x}; \theta) \right) + \frac{w_B}{|X(\partial\Omega)|} \sum_{\mathbf{x} \in X(\partial\Omega)} (U(\mathbf{x}; \theta) - g(\mathbf{x}))^2. \quad (2.7)$$

Here, again, the boundary condition is enforced with the L^2 -integral of the error, $U(\mathbf{x}; \theta) - g(\mathbf{x})$, and the integrals of the energy term and the boundary condition term are approximated by the Monte–Carlo method; the weight factors w_I and w_B are analogous to those in the PINN loss.

The integral form of the deep Ritz loss function, before approximation via numerical integration, reads

$$J_R(\theta) := w_I \int_{\Omega} \left(\frac{1}{2} |\nabla U(\mathbf{x}; \theta)|^2 - f(\mathbf{x})U(\mathbf{x}; \theta) \right) d\mathbf{x} + w_B \int_{\partial\Omega} (U(\mathbf{x}; \theta) - g(\mathbf{x}))^2 ds(\mathbf{x}). \quad (2.8)$$

The subscript R indicates that the loss is formed by the deep Ritz formulation. In addition, the subscript M in the loss $J_{R,M}(\theta)$ in Eq. (2.7) means that the integral in the deep Ritz loss $J_R(\theta)$ in Eq. (2.8) is approximated by the Monte–Carlo method.

When the boundary condition is implemented as hard constraints using ansatz functions, that is, using the neural network function $\tilde{U}(\mathbf{x}; \theta)$ in Eq. (2.6), we obtain the integral loss function

$$J_{R_I}(\theta) := \int_{\Omega} \left(\frac{1}{2} |\nabla \tilde{U}(\mathbf{x}; \theta)|^2 - f(\mathbf{x})\tilde{U}(\mathbf{x}; \theta) \right) d\mathbf{x}$$

and train the neural network $\tilde{U}(\mathbf{x}; \theta)$ for the loss $J_{R_I,M}(\theta)$ by approximating the integral in J_{R_I} using the Monte–Carlo method.

It has been numerically studied that for the Poisson model problem, the trained solution $U(\mathbf{x}; \theta_P)$ with the PINN loss $J_{P,M}(\theta)$ gives better training results than the trained solution $U(\mathbf{x}; \theta_R)$ with the deep Ritz loss $J_{R,M}(\theta)$; see [40]. In our work, we will reinvestigate the performance of the two approaches for various hyper parameter settings and report some of our new findings.

As an enhancement to soft enforcement of boundary conditions, an augmented Lagrangian term can be included to the loss function [42] to obtain,

$$L_{P,M}(\theta, \lambda) := J_{P,M}(\theta) + \frac{1}{|X(\partial\Omega)|} \sum_{\mathbf{x} \in X(\partial\Omega)} (U(\mathbf{x}; \theta) - g(\mathbf{x}))\lambda(\mathbf{x})$$

and

$$L_{R,M}(\theta, \lambda) := J_{R,M}(\theta) + \frac{1}{|X(\partial\Omega)|} \sum_{\mathbf{x} \in X(\partial\Omega)} (U(\mathbf{x}; \theta) - g(\mathbf{x}))\lambda(\mathbf{x}),$$

for the PINN and deep Ritz loss functions, respectively. Here, the boundary condition is enforced as constraints on the neural network solution $U(\mathbf{x}; \theta)$ by introducing Lagrange multipliers $\lambda(\mathbf{x})$ for each collocation point $\mathbf{x}$ in the training sampling set $X(\partial\Omega)$. Hence, $\lambda(\mathbf{x})$ are additional parameters that have to be trained, in addition to θ . The use of such an augmented Lagrangian term can improve slow training progress for the boundary loss term and can provide a more accurate trained neural network solution, $U(x; \theta)$. In the augmented Lagrangian approach, the parameters θ and λ are then optimized for the PINN and deep Ritz loss functions in the following sense:

$$(\theta_P, \lambda_P) := \arg \left(\max_{\lambda} \min_{\theta} L_{P,M}(\theta, \lambda) \right) \quad \text{resp.} \quad (\theta_R, \lambda_R) := \arg \left(\max_{\lambda} \min_{\theta} L_{R,M}(\theta, \lambda) \right).$$

We note that the above optimization problems for θ are non-linear and non-convex while those for λ are linear. We thus use the Adam optimization method [20] in the gradient update for θ with a small learning rate ϵ and a simple gradient update for λ with a learning rate α , i.e.,

$$\lambda = \lambda + \alpha \nabla_{\lambda} L_{P,M} \quad \text{or} \quad \lambda = \lambda + \alpha \nabla_{\lambda} L_{R,M}.$$

The learning rate α is often set to a larger value than the learning rate ϵ , as proposed in two-scale update schemes for min-max optimization problems; see [14, 25, 7]. In our numerical experiments, we set $\epsilon = 0.001$ for the Adam optimizer and $\alpha = 1$ for the gradient ascent update.

We note that the augmented Lagrangian method can be considered as a loss balancing scheme, and in our numerical experiments, we will also conduct comparisons on various loss balancing schemes as listed in Table 1.